PlateIdeas – Product Requirements Document

Hackathon project | Build window: 50 minutes | Prepared for: Team Lead

1. Overview

PlateIdeas is a single-page meal suggestion app. A user enters the ingredients they already have, and the app suggests meals they can cook, gives a step-by-step recipe for each, and links to cooking videos for every suggested meal.

One-line pitch: Tell us what's in your kitchen, and we'll tell you what to cook and show you how.

2. Problem Statement

People often look at the ingredients they have and cannot decide what to cook. This leads to wasted food, unnecessary spending on takeaway or extra groceries, and daily decision fatigue. Existing recipe sites make users search by dish name, not by what they actually have.

3. Goals and Non-Goals

Goals

- Let a user go from ingredients to meal ideas in under 30 seconds.
- Show a complete recipe for each suggested meal.
- Provide a working video recommendation for each meal.
- Deliver a polished, demo-ready flow within the 50-minute build window.

Non-Goals (for this build)

- User accounts or login.
- A database or persistent cloud storage.
- Grocery delivery or ordering integration.
- Nutrition or calorie tracking.
- Native mobile apps.

4. Target Users

- **Students and young professionals** cooking on a budget with limited ingredients.
- **Busy households** who want fast meal decisions.
- **Beginner cooks** who need guided steps and video support.

5. User Flow

1. User opens the app and sees an ingredient input with quick-add chips (rice, eggs, tomatoes, onions, chicken, beans, pasta, potatoes).
2. User types ingredients separated by commas or taps chips to add them.
3. User clicks **Suggest meals**.
4. A loading state appears while the AI generates suggestions.
5. The app displays 4 meal cards.
6. User reads the recipe and clicks **Watch video** to open a YouTube search for that dish in a new tab.

6. Functional Requirements

Must Have (MVP)

ID	Requirement
F1	Text input for ingredients, comma separated.
F2	Clickable ingredient chips that add items to the input.
F3	"Suggest meals" button that calls an LLM API.
F4	Display 4 meal cards, each showing: name, short description, cook time, ingredients the user has, ingredients missing, and numbered recipe steps.
F5	"Watch video" button on each card that opens a YouTube search link built from the meal's video search query.
F6	Loading spinner while the request is in progress.
F7	Error handling with a friendly message if the API call fails or returns invalid data.

ID	Requirement
F8	Assume basic pantry items (salt, oil, water, pepper) are always available.

Should Have (if time allows)

ID	Requirement
F9	Diet filter: vegetarian, low budget, quick (under 20 minutes).
F10	Local cuisine option (e.g. Nigerian dishes).
F11	"Save to weekly plan" button with a simple 7-day grid held in memory. This turns the app from a meal suggester into a meal planner.

Could Have (stretch)

- Real video thumbnails using the YouTube Data API.
- Shopping list generated from the missing ingredients.
- Shareable meal plan.

7. Video Recommendation Approach

LLMs tend to invent YouTube URLs that do not work. To avoid broken links, the AI returns only a `video_search_query` for each meal (for example, "how to cook jollof rice with chicken"). The app builds the link in code:

`https://www.youtube.com/results?search_query=` + URL-encoded query

This needs no API key and cannot break. It can be upgraded to the YouTube Data API later.

8. AI Output Contract

The LLM must return JSON only, in this shape:

```
{
  "meals": [
    {
      "name": "",
      "description": "",
      "cook_time_minutes": 0,
      "ingredients_you_have": [],
      "ingredients_missing": [],
      "steps": [],
      "video_search_query": ""
    }
  ]
}
```

The app must strip any markdown code fences before parsing the response.

9. Technical Approach

- **Frontend:** plain HTML, CSS and JavaScript in a single page, with no framework.
- **AI:** an OpenAI-compatible LLM API (a free tier such as Groq or Gemini is recommended) called with `fetch`.
- **Build tooling:** OpenCode to scaffold the application from a single prompt.
- **Storage:** none for the MVP. The weekly plan, if built, is held in memory only.
- **Design:** warm, modern food-app look with a soft orange accent and white rounded cards.

10. Non-Functional Requirements

- Results should appear within roughly 10 seconds of clicking the button.
- The layout should work on both desktop and mobile screens.
- The API key is held in a constant for the demo and must not be committed to a public repository.

11. Timeline (50 Minutes)

Time	Activity
0-5 min	Split roles and confirm scope.
5-15 min	Scaffold the app with OpenCode.

Time	Activity
15-30 min	Get the LLM call and meal cards working.
30-40 min	Add video links, loading state and error handling.
40-45 min	Test with three ingredient sets, including an unusual one (e.g. eggs, rice, tomatoes).
45-50 min	Rehearse the demo. No new features.

12. Team Roles

- **UI/Frontend:** layout, cards, chips, styling.
- **AI/Integration:** API call, prompt, JSON parsing, error handling.
- **Demo and Pitch:** test cases, demo script, backup recording, presentation.

13. Risks and Mitigations

Risk	Mitigation
LLM returns invalid JSON	Strip code fences, wrap parsing in try/catch, show a friendly retry message.
API fails or is rate limited during the demo	Keep a screen recording or screenshots as a backup.
Invented video links	Use YouTube search links built in code (see section 7).
Scope creep	Freeze features at the 40-minute mark.
Slow responses	Show a loading spinner and keep the prompt concise.

14. Success Criteria

- A user can enter ingredients and receive 4 meal suggestions with recipes end to end.
- Every meal card has a working video link.
- The live demo runs without errors using at least two different ingredient sets.
- The product can be explained and demonstrated within 60 seconds.

15. Demo Plan (60 Seconds)

1. **Problem (10 sec):** people stare at their fridge, waste food and overspend.
2. **Live demo (40 sec):** enter four ingredients, click Suggest meals, open a recipe and click Watch video.
3. **Impact (10 sec):** less food waste, less money spent, less decision fatigue.